Section 03: TypeScript Deep Dive

0. TypeScript とは？

「静的型付け」で進化する JavaScript

TypeScript と JavaScript の決定的な違いは、**「いつその型をチェックするか」** です。

- **JavaScript（動的型付け）**：
  - 実行してエラーが出るまで、型の間違いに気づけません。
- **TypeScript（静的型付け）**：
  - **実行する前（コードを書き込んでいる最中）** に、エディタが型をチェックして間違いを教えてくれます。

基本の式: `TypeScript = JavaScript + 静的型付け（事前の型チェック）`

TypeScript の仕組みとメリット

- **動作の仕組み**: ブラウザは直接理解できないため、実行前に **JavaScript に翻訳（トランスパイル）** して使います。
- **安心の開発**: 「実行したら動かない」というミスを、書いている瞬間に防げることがあります。
- **強力な補完**: オブジェクトの中身をエディタが覚えているので、タイピングが劇的に楽になります。

【Column】 コンパイル と トランスパイル

「コードを変換する」仕組みには厳密には2つの呼び方があります。

- **コンパイル (Compile)**
  人間が読めるコード（C言語など）から、**機械だけが読めるコード（0と1の機械語）など、抽象度の全く違う別の次元へ変換する** こと。
- **トランスパイル (Transpile)**
  人間が読めるコード（TypeScript）から、**人間が読める別のコード（JavaScript）など、同じ抽象度のまま別の言語へ変換する** こと。（Transform + Compile）

> ※ 現場ではどちらもひっくるめて「コンパイルする」「ビルドする」と呼ばれることが多いですが、技術的には TypeScript ➡️ JavaScript は「トランスパイル」です！

1. 復習：JSプログラミングにおける「型」とは？

データの「種類」を定義するルール

これまで変数や定数を学んできましたが、プログラムが扱うすべてのデータには「型（Type）」というデータの種類が決まっています。

- `string` **(文字列)**: 名前やメッセージなど（`"Shake"`）
- `number` **(数値)**: 金額やカウントなど（`123`, `0.5`）
- `boolean` **(真偽値)**: はい/いいえの状態（`true`, `false`）
- `Array / Object`: 複数のデータをまとめたもの

「型」が必要な理由:
「名前（文字列）に 10 を足す」といった、人間から見たら明らかに **「意味の通らない操作」** を、コンピュータが事前（実行前）に見つけるための手がかりになります。

2. なぜ TypeScript を使うのか？

型がない世界のリスク

JavaScript は非常に自由ですが、その自由さがバグの原因になります。

- **実行するまでエラーがわからない**:
  関数に文字列を渡すべきところに数値を渡しても、動かしてみるまで気づけません。
- **補完が効かない**:
  オブジェクトにどんなプロパティがあるか、記憶に頼るしかありません。
- **大規模開発での混乱**:
  他人が書いたコード（あるいは1ヶ月前の自分が書いたコード）の使い方がわからなくなることがあります。

TypeScript がもたらす「安心感」

TypeScript は JavaScript に **Static Typing（静的型付け）** を追加した言語です。

- **静的解析**: コードを実行する前に、エディタがミスを「赤線」で指摘してくれます。
- **強力なオートコンプリート**: オブジェクトが持つプロパティを推測して表示してくれます。

JavaScript と TypeScript の違い

TypeScript Flow

JavaScript の場合

```
// JavaScript の場合：実行するまでバグに気づけない
function buy(price) {
  return price * 1.1;
}
buy("1,000円"); // NaN（型エラー）が返るが、書いている最中は気づけない
```

TypeScript の場合

```
// TypeScript の場合：書いている途中でエディタが怒ってくれる
function safeBuy(price: number): number {
  return price * 1.1;
}
safeBuy("1,000円");
//      ^^^^^^^^^
// Argument of type 'string' is not assignable to parameter of type 'number'.
```

強力な補完（オートコンプリート）

```
const user = { name: "Shake", age: 20 };

user.na.. // ここまで打つだけで "name" が候補に出てくる！
// 記憶に頼る必要がなく、タイポ（打ち間違い）も防げます。
```

TypeScript の基本的な書き方：型注釈 (Type Annotation)

変数や関数の後ろに `: 型名` を添えて、データの種類を明示的に指定することを **「型注釈（Type Annotation）」** と呼びます。

```
// 変数に型をつける
const userName: string = "Shake";
const price: number = 1000;

// 関数の引数と戻り値に型をつける
function greet(name: string): string {
  return `Hello, ${name}`;
}
```

TypeScript はこの「注釈」をもとに、正しいデータが扱われているかをチェックし、エディタでミスを教えてくれるようになります。

理解度チェック：TS の基本（問題）

次の JavaScript コードを TypeScript に書き換えて、引数と戻り値に型をつけてください。
なお、引数の `text` は文字列を受け取ることを、戻り値は数値を返すことを示してください。

```
function getLength(text) {
  return text.length;
}
```

- ヒント：length プロパティは文字列の長さを表すプロパティです。

理解度チェック：TS の基本（答え）

```
function getLength(text: string): number {
  return text.length;
}
```

- 文字列を受け取って、その長さを数値で返すので `string` 型と `number` 型を指定します。

3. 型推論と特別な型 (any / unknown)

型推論：賢い TypeScript

実は、すべての変数に型を細かく書く必要はありません。
TypeScript は代入された値から「これは数値だ」「これは文字列だ」と自動で判断してくれます。これを **型推論（Type Inference）** と呼びます。

```
let count = 10; // 自動的に number 型として扱われる
count = "10"; // 型推論のおかげで、ここでエラーを出してくれる
```

実務のコツ:
基本は「推論」に任せ、関数の引数や、中身が空で始まる変数など、推論できない場所にだけ「型注釈」を書くのがモダンで綺麗なコードの書き方です。

any 型：すべてを許容する「禁じ手」

`any` 型を指定すると、その変数に対する **TypeScript の型チェックがすべて無効化されます。**

- **何でもできる**: 文字列を入れても数値を入れても怒られません。存在しないメソッドを呼び出してもエラーになりません。
- **最大のリスク**: TypeScript を使うメリット（エラーの事前検知、補完）がすべて失われます。「書いている時はエラーがないのに、実行するとクラッシュする」という最悪の状態を招きます。
- **合言葉**: **「基本的に `any` は使わない」**。どうしても型がわからない時の一時的な避難場所以外では使いません。

```
let value: any = "Shake";

value = 123; // OK（型のエラーが出ないが、どこかで不具合が起きる可能性）
value.push(); // OK（トランスパイルは通る）... 実際には数値に push はできないので実行時にエラーになる！
```

代替案：より安全な unknown 型

どうしても型が分からない場合、現代のTypeScriptでは `any` ではなく `unknown` 型を使うのがベストプラクティスです。

- **`unknown` の特徴**: 「何が入っているか分からない」という事実は同じですが、**「中身を特定（型の絞り込み）するまでは、勝手な操作を許さない」** という安全装置が働きます。
- **実務での扱い**: APIから取得した得体の知れないデータなどは、まず `unknown` で受け取り、後述するバリデーション（zodなど）を通して安全な型に変換してから使います。

```
let safeValue: unknown = "Shake";

// safeValue.push(); // エラー！「よくわからないものに push はできません」と怒ってくれる

if (typeof safeValue === "string") {
  // ここからは string として扱える（型の絞り込み）
  console.log(safeValue.length);
}
```

エラーを強制的に黙らせるコメント

どうしても型エラーが解決できない、あるいはライブラリ側(他人が作ったコード)の型定義が間違っている場合に、**コンパイルエラーを無理やり消す**ための特殊なコメントがあります。

- `// @ts-ignore` **「以下を強制的に無視せよ」**:
  次の行で発生する型エラーを**無条件で無視**します。
  - **リスク**: コードを修正してエラーが消えた後も「無視し続ける」ため、将来その行に**別の本当のバグ**が混入しても気づけなくなります。
- `// @ts-expect-error` **「以下はエラーなので無視せよ」**:
  次の行で「エラーが出るはずだ」と宣言して黙らせます。
  - **メリット**: もしコードが修正され、エラーが出なくなった場合、**「エラーが出るはずなのに出ていないぞ」と TypeScript が逆にエラーを出してくれます。** 不要になったコメントを消し忘れることがありません。

```
// 例：外部ライブラリの型定義が古く、実際には存在する「os」プロパティがエラーになる場合

// @ts-ignore
// とにかくエラーを消す（もしライブラリが更新されても気づけない）
console.log(config.os);

// @ts-expect-error
// 「エラーが出るはず」と宣言（ライブラリが更新されて型が正しくなれば、ここがエラーになり気づける）
console.log(config.os);
```

結論: どちらも「禁じ手」ですが、どうしても使うなら `@ts-expect-error` を選びましょう。

理解度チェック：型定義と any（問題）

以下のコードで、TypeScript がエラー（赤線）を出すのはどの行でしょうか？

```
let price: any = 1000;
price = "無料";

function update(p: number) {
  console.log(p);
}

update(price); // (A)
update("0"); // (B)
```

理解度チェック：型定義と any（答え）

答え：(B)

- `price` は `any` 型なので、(A) のように「数値が必要な関数」に渡してもエラーになりません（これが any の危険なところです）。
- (B) は「文字列」を直接「数値型」の引数に渡しているため、明確に型エラーになります。

4. オブジェクトの型定義：Interface と Type

Interface (インターフェース)

オブジェクト（複数のデータがまとまったもの）が、**「どのようなプロパティを持ち、それぞれの値が何型か」という「設計図（形）」** を定義します。

- **一貫性の保証**: 同じインターフェースを使うことで、複数の箇所で同じ形状のデータを扱うことが保証されます。
- **過不足のチェック**: 定義したプロパティが足りなかったり、余計なプロパティが入っているとエラーになります。
- **読みやすさ**: 何でも入る `Object` 型ではなく、具体的で名前のついた型（例: `User`）として扱えます。

Interfaceの例

```
interface User {
  id: string;
  name: string;
  email: string;
  // プロパティ名の後ろに ? をつけると「あってもなくても良い（任意）」になります
  age?: number;
}

// OK: 全ての必須項目が揃っている
const staff: User = {
  id: "st-01",
  name: "Taro",
  email: "taro@example.com",
};

// NG: name が欠けているためエラーになる
const visitor: User = {
  id: "v-02",
  email: "visitor@example.com",
};
```

【課題 1】実行委員スタッフの管理 (Lv.1)

Q1. 次の条件のうち、「あってもなくても良い（任意）」プロパティを定義する記号はどれでしょうか？

A) `!`
B) `?`
C) `:`

Q2. 以下の条件をすべて満たす `Staff` インターフェースの定義として正しいものはどれでしょうか？

1. `id`: 数値 (必須)
2. `name`: 文字列 (必須)
3. `isLeader`: 真偽値 (必須)
4. `phoneNumber`: 文字列 (任意)

A)

```typescript
interface Staff {
  id: string;
  name: string;
  isLeader: boolean;
  phoneNumber: string;
}
```

B)

```typescript
interface Staff {
  id: number;
  name: string;
  isLeader: boolean;
  phoneNumber?: string;
}
```

【ヒント】

- プロパティ名の後ろに `?` をつけると、その項目は省略可能（あってもなくても良い）になります。
- 真偽値は `boolean` 型、数値は `number` 型を使います。

【解答】

正解： Q1: B / Q2: B

```typescript
interface Staff {
  id: number;
  name: string;
  email: string;
  phoneNumber?: string; // Optional
  isLeader: boolean;
}

const staff: Staff = {
  id: 101,
  name: "Shake",
  email: "shake@example.com",
  isLeader: true,
  // phoneNumber はなくてもOK
};
```

Interface で「機能（メソッド）」を定義する

オブジェクトには「データ」だけでなく、**「何ができるか（関数/メソッド）」** も含めることができます。

- **メソッドの定式**: `メソッド名(引数: 型): 戻り値の型;`
- **void 型**: 「値を何も返さない（`return` しない）」ことを表す特別な型です。コンソールに表示するだけ、などの時に使います。

```
interface Staff {
  id: number;
  name: string;
  // 「こんにちは」と挨拶するメソッド（戻り値なし）
  sayHello(): void;
}

const staff: Staff = {
  id: 101,
  name: "Shake",
  sayHello() {
    console.log(`Hi, I'm ${this.name}!`);
  },
};

staff.sayHello(); //呼び出す
```

Interface の継承 (`extends` キーワード)

既存のインターフェースをベースに、新しい項目を追加した「拡張版」の型を作ることができます。

- **再利用性**: 共通の項目（`id`, `name` 等）を何度も書く必要がなくなります（DRY原則）。
- **階層構造**: 「ユーザー」という基本型があり、それを拡張した「実行委員ユーザー」といった関係性を表現できます。

Interface の継承

```typescript
// 基本となる型
interface User {
  id: string;
  name: string;
}

// User の中身を全て引き継ぎ、さらに department を追加
interface StaffUser extends User {
  department: string;
  role: "admin" | "member";
}

const myProfile: StaffUser = {
  id: "st-77",
  name: "Shake",
  department: "IT部",
  role: "admin",
};
```

練習問題: オプショナルなプロパティ

オブジェクトのプロパティで、 **あってもなくても良い（任意）** なものを定義したいです。
プロパティ名の後ろに付ける記号として正しいものはどれですか？

A) `!`
B) `?`
C) `:`

```
interface User {
  id: number;
  name: string;
  // bio（自己紹介）は任意にしたい
  bio____: string;
}
```

【解説】 オプショナルなプロパティ

```
interface User {
  id: number;
  name: string;
  bio?: string;
}
```

`?` をプロパティ名の後ろに付けると、そのプロパティは「省略可能」になります。

Type Alias (型別名)

既存の型や、複数の型を組み合わせたものに **「独自の名前（別名）」** をつけることができます。

- **あだ名をつける**: 複雑で長い型に短い名前をつけて、コードの意図を明確にします。
- **組み合わせの定義**: 「A または B」といった柔軟な型の組み合わせが得意です。
- **用途**: オブジェクトだけでなく、特定の「決まった値のリスト」を作る際によく使われます。

```typescript
// 1. Union Types（共用体型）： 複数の型の「どちらか」
type ID = string | number;

// 2. Literal Types（リテラル型）： 特定の「値」そのものを型にする
//  「この3つの文字列以外は認めない」という強力な制約をかけられます
type Status = "pending" | "active" | "closed";

const userId: ID = "u123"; // OK
const currentStatus: Status = "active"; // OK

// NG! 定義にない文字列を入れようとするとエラーになる
const invalidStatus: Status = "done";
```

練習問題: Union型 (共用体型)

変数が「文字列」または「数値」の**どちらか**の値を取りうる場合、どのように定義しますか？
間に挟む記号として正しいものはどれですか？

A) `&`
B) `|`
C) `,`

```
// IDは文字列も数値も許容したい
## 【解説】 Union型（共用体型）

```typescript
type ID = string | number;
```
```

`|` (パイプ) を使うと「A または B」という型（Union型）を定義できます。
これを **Union Types** と呼びます。

練習問題: 継承とLiteral Types

以下の要件を満たすコードとして、正しいものはどれですか？

1. `EventItem` を継承して `StagePerformance` を作る
2. `status` には "waiting", "performing", "finished" の3つの文字列のみを許可する

Q1. 継承の書き方として正しいものは？

A) `interface StagePerformance : EventItem`

B) `interface StagePerformance extends EventItem`

C) `interface StagePerformance implements EventItem`

Q2. status の型定義として正しいものは？

A) `status: "waiting" | "performing" | "finished"`

B) `status: ["waiting", "performing", "finished"]`

C) `status: string` (何でも入るようにする)

【ヒント】

- 継承には `extends` を使います。
- `status` のように決まった選択肢は **Literal Types**（文字列を `|` で繋ぐ）を Type Alias または直接指定して定義します。

【解答】 継承とLiteral Types

Q1の答え：B (extends)

Q2の答え：A ("waiting" | ...)

```
interface EventItem {
  id: string;
  title: string;
}

// Literal Types を使った状態定義
type PerformanceStatus = "waiting" | "performing" | "finished";

interface StagePerformance extends EventItem {
  startTime: string;
  status: PerformanceStatus;
}

const opening: StagePerformance = {
  id: "stg-01",
  title: "オープニングセレモニー",
  startTime: "10:00",
  status: "performing",
};
```

Interface と Type Alias の使い分け

どちらも「型」を作るために使われますが、得意分野が異なります。

Interface は「拡張可能な設計図」

- **継承が得意**: 既存の型をベースに一部を追加した新しい型を作るのが得意です。
- **オブジェクト専用**: インターフェースは「オブジェクトの形」を定義するための専用機能です。
- **推奨**: ユーザー、記事、ブース情報など、**「独立したモノ（実体）」** を定義するときに使います。

Type Alias は「柔軟なあだ名」

- **組み合わせが得意**: `string | number` のように、複数の型を合体させたものは Type Alias でしか名前をつけられません。
- **リテラル型の定義**: `"pending" | "active"` のような「特定の選択肢のセット」を定義するのに最適です。
- **推奨**: ステータス、IDの種類、設定値など、**「値のパターン」** を定義するときに使います。

結論:
最初は「オブジェクトなら **Interface**、それ以外（選択肢/複雑な合体）なら **Type Alias**」と使い分ければOK。

> **昨今の実務トレンド**:
> React コミュニティなどを中心に、意図しない型のマージ（同名 Interface の自動結合）を防ぐなどの理由から **「プロジェクト内の型定義はすべて `Type Alias` (type) で統一する」** という現場も増えています。
> 本研修では歴史的経緯を踏まえて両方紹介しますが、**「迷ったら `type` を使う」** と割り切ってしまっても構いません。

理解度チェック：Interface と Type（問題）

次のコードの `( ? )` に入る適切なキーワードは何でしょうか？

A) `implements`

B) `extends`

C) `import`

```
interface Animal {
  name: string;
}

interface Dog ( ? ) Animal {
  bark(): void;
}
```

理解度チェック：Interface と Type（答え）

答え：B (extends)

- Interface を拡張して新しい型を作るには `extends` キーワードを使います。

5. 安全なデータ操作（?. と ??）

Optional Chaining (`?.`)

データの存在が不確かな場合に、プログラムを停止（クラッシュ）させずに安全にアクセスする仕組みです。

- **問題点**: `user.profile.age` と書いたとき、もし `profile` が `null` だとエラーが発生し、アプリ全体が止まってしまいます。
- **解決策**: `?.` を使うと、途中のデータがなくても **「エラーにならずに undefined を返す」** だけになり、安全です。

```
// もし profile がなくてもクラッシュしない
const age = user.profile?.age;

// 実行結果のイメージ
// user.profile がある場合   -> 20
// user.profile がない場合   -> undefined（エラーにならない！）
```

練習問題: Optional Chaining

深くネストされたプロパティにアクセスする際、途中のオブジェクトが存在しない（`null` や `undefined`）可能性があります。
エラーを起こさずに安全にアクセスできる選択肢はどれでしょうか？

A) `user.profile.address.city`
B) `user.profile?.address?.city`
C) `user.profile!address!city`

```
const user = { profile: undefined };

// profile がなくてもクラッシュしないようにする
const city = ( ? );
```

【解説】 Optional Chaining

答え：B

```
const city = user.profile?.address?.city;
```

`?.` を使うと、左側が `null` や `undefined` の場合、即座に `undefined` を返して終了します。これによりランタイムエラーを防げます。

Nullish Coalescing (`??`)

データが `null` や `undefined` だった場合に、**「デフォルト値（予備の値）」** を設定する仕組みです。

- **用途**: 設定値がない場合に初期値を割り当てる際などに頻繁に使います。
- **実例**: ユーザーが未入力だった項目の補填などに活躍します。

```
// ?. と ?? を組み合わせた最強の形
const theme = user.settings?.theme ?? "light";

// 1. settings.theme が "dark" なら -> "dark"
// 2. settings がない、または theme がないなら -> "light"（予備が使われる）
```

理解度チェック：安全なアクセス（問題）

次のコードを実行したとき、`result` の値は何になるでしょうか？

A) `null`
B) `undefined`
C) `"default-ui"`

```
const user = { name: "Shake", settings: null };

const result = user.settings?.theme ?? "default-ui";
```

理解度チェック：安全なアクセス（答え）

答え：C ("default-ui")

- `user.settings` が `null` なので、`user.settings?.theme` は `undefined` を返します。
- `??`（Nullish Coalescing）は左側が `null/undefined` のときに右側を返すので、最終的に `"default-ui"` が代入されます。

6. ジェネリクス (Generics)

定義時に型を確定させない仕組み

ジェネリクス（Generics） とは、型そのものを「引数」として扱い、定義段階では確定させずに**使用時に中身の型を指定する**仕組みです。

1. なぜ必要なのか（非効率な例）

データの枠組みは同じなのに、中身の型が違うだけで似たような定義を何度も作らなければならない不都合を解消します。

```
interface StringRes {
  data: string;
  status: number;
}
interface NumberRes {
  data: number;
  status: number;
}
// ... 型が増えるたびに新しい定義が必要になる（非効率）
```

2. ジェネリクスによる解決

変化する部分に `T` という **「型引数（プレースホルダ）」** を置き、**「後で型が入る空欄」** を作ります。

- **定義**: `interface ApiResponse<T> { data: T; }` （T は型が入るための変数のようなもの）
- **置換**: `ApiResponse<string>` と使うと、定義内の `T` が一斉に `string` に置き換わります。

例

```
// T という「型を受け取るための変数」を用意した定義
interface ApiResponse<T> {
  data: T;
  status: number;
}

// 使用時に T の部分へ具体的な型（string）を流し込む
const textRes: ApiResponse<string> = { data: "success", status: 200 };

// 使用時に T の部分へ別の型（User）を流し込む
const userRes: ApiResponse<User> = {
  data: { id: "1", name: "Alpha", email: "a@example.com" },
  status: 200,
};
```

理解度チェック：ジェネリクス（問題）

次の型定義を元に、数値(`number`)をデータとして持つ `Item` 型の変数を宣言する場合、正しい書き方はどれでしょうか？

A) `const myItem: Item = { id: "1", content: 100 };`

B) `const myItem: Item<number> = { id: "1", content: 100 };`

C) `const myItem: Item(number) = { id: "1", content: 100 };`

```
interface Item<T> {
  id: string;
  content: T;
}
```

理解度チェック：ジェネリクス（答え）

答え：B

```
const myItem: Item<number> = {
  id: "item-001",
  content: 100,
};
```

- `<T>` の部分に `number` を流し込むことで、`content` プロパティが数値型として確定します。

【課題】汎用的な API レスポンス (Lv.3)

「異なる種類のデータ」を包む共通のレスポンス型をジェネリクスで作成してください。

1. **ジェネリック型** `ApiResponse<T>`: 「`data`: T型」と「`status`: 数値」を持つ。
2. **変数 1**: `ApiResponse<string>` を使って、文字列データを返す変数を作成。
3. **変数 2**: `ApiResponse<User>` を使って、前述の `User` 型を返す変数を作成。

【課題：ヒント】

- インターフェース名の後ろに `<T>` をつけ、中身で `T` を型として使用します。

【課題：解答例】

```typescript
interface ApiResponse<T> {
  data: T;
  status: number;
}

const textResponse: ApiResponse<string> = {
  data: "成功しました",
  status: 200,
};

const userResponse: ApiResponse<User> = {
  data: { id: 1, name: "Shake" },
  status: 200,
};
```

まとめ

本セクションのまとめ

- **TypeScript の本質**: JavaScript の自由さに「型」という制約を加え、**実行する前にミスを機械的に見つける**ためのツール。
- **型アノテーション**: `: string` のように型を明示することで、エディタの強力な補完と保護を受けられる。
- **Interface と継承**: 「物の形」を定義し、`extends` で再利用・拡張することで共通化を促進する。
- **Type Alias の柔軟性**: リテラル型（特定の文字列のみ等）や Union Types を使い、**「不正な状態」を型レベルで発生させない**設計ができる。
- **安全なデータアクセス**: `?.` (Optional Chaining) と `??` (Nullish Coalescing) を使い、不確実なデータに対してもクラッシュしない堅牢なコードを書く。
- **ジェネリクス**: 型をパラメータ化（`<T>`）し、構造は同じだが中身が異なるデータ（API レスポンス等）を効率的かつ安全に定義する。
- **「型は仕様書」**: 型を正しく定義することは、未来の自分やチームメンバーに「このデータはどう扱うべきか」を伝える最も正確なドキュメントになる。

次は、この型安全な環境を活かして「Hono / バックエンド開発」の基礎を学びます。